

Bài tập lập trình: Tích hợp OBB (Oriented Bounding Box) Tree vào thư viện Chai3D

1. Mục tiêu bài tập

Bài tập này nhằm giúp sinh viên hiểu rõ và thực hành cài đặt thuật toán phát hiện va chạm (Collision Detection) sử dụng cấu trúc dữ liệu OBB Tree trong không gian 3D. Sinh viên sẽ mở rộng thư viện mã nguồn mở Chai3D bằng cách thêm vào một module phát hiện va chạm mới dựa trên OBB, tương tự như các module AABB (Axis-Aligned Bounding Box) hay Bounding Sphere đã có sẵn.

Sau khi hoàn thành bài tập, sinh viên sẽ có khả năng:

- Hiểu và áp dụng phương pháp Bounding Volume Hierarchy (BVH) để tăng tốc độ phát hiện va chạm.
- Nắm vững thuật toán xây dựng OBB cho một tập hợp các đa giác (triangles).
- Thực hiện thuật toán duyệt cây (Tree Traversal) để kiểm tra va chạm giữa công cụ haptic (dạng đoạn thẳng/tia) và đối tượng 3D.
- Lập trình hướng đối tượng (C++) để mở rộng một thư viện có sẵn (Chai3D).

Lưu ý về quy trình kiểm tra va chạm:

- Coarse Test: Kiểm tra va chạm thô giữa công cụ (haptic device) và các khối bao (OBB) của cây phân cấp, giúp nhanh chóng loại bỏ các vùng không va chạm.
- Fine Test: Khi phát hiện va chạm tại node lá, thực hiện kiểm tra chi tiết giữa công cụ và các tam giác (triangles) thực tế để xác định chính xác điểm va chạm.

2. Mô tả yêu cầu

Thư viện Chai3D hiện tại hỗ trợ các phương pháp phát hiện va chạm như AABB Tree, Bounding Sphere Tree và Brute-force. Nhiệm vụ của bạn là tạo ra các lớp (classes) mới để hỗ trợ OBB Tree.

Cụ thể, bạn cần thực hiện các công việc sau:

1. **Định nghĩa cấu trúc OBB:** Tạo lớp `cCollisionOBBox` để lưu trữ thông tin của một OBB (tâm, các trục tọa độ địa phương, độ dài các nửa trục).
2. **Xây dựng cây OBB (Top-down approach hoặc Bottom-up approach):** Tạo các lớp kế thừa từ cấu trúc node của Chai3D (hoặc tự định nghĩa `cCollisionOBBNode`, `cCollisionOBBLeaf`, `cCollisionOBBInternal`) để xây dựng cây nhị phân OBB từ một mesh (tập hợp các tam giác).
3. **Thuật toán sinh OBB (Gợi ý sử dụng PCA):** Tính toán OBB tối ưu hoặc gần tối ưu bao

quanh một tập hợp các tam giác tại mỗi node. Bạn được khuyến khích sử dụng Phân tích thành phần chính (PCA - Principal Component Analysis) để tìm các trục (eigenvectors) của OBB dựa trên phân bố của các đỉnh tam giác.

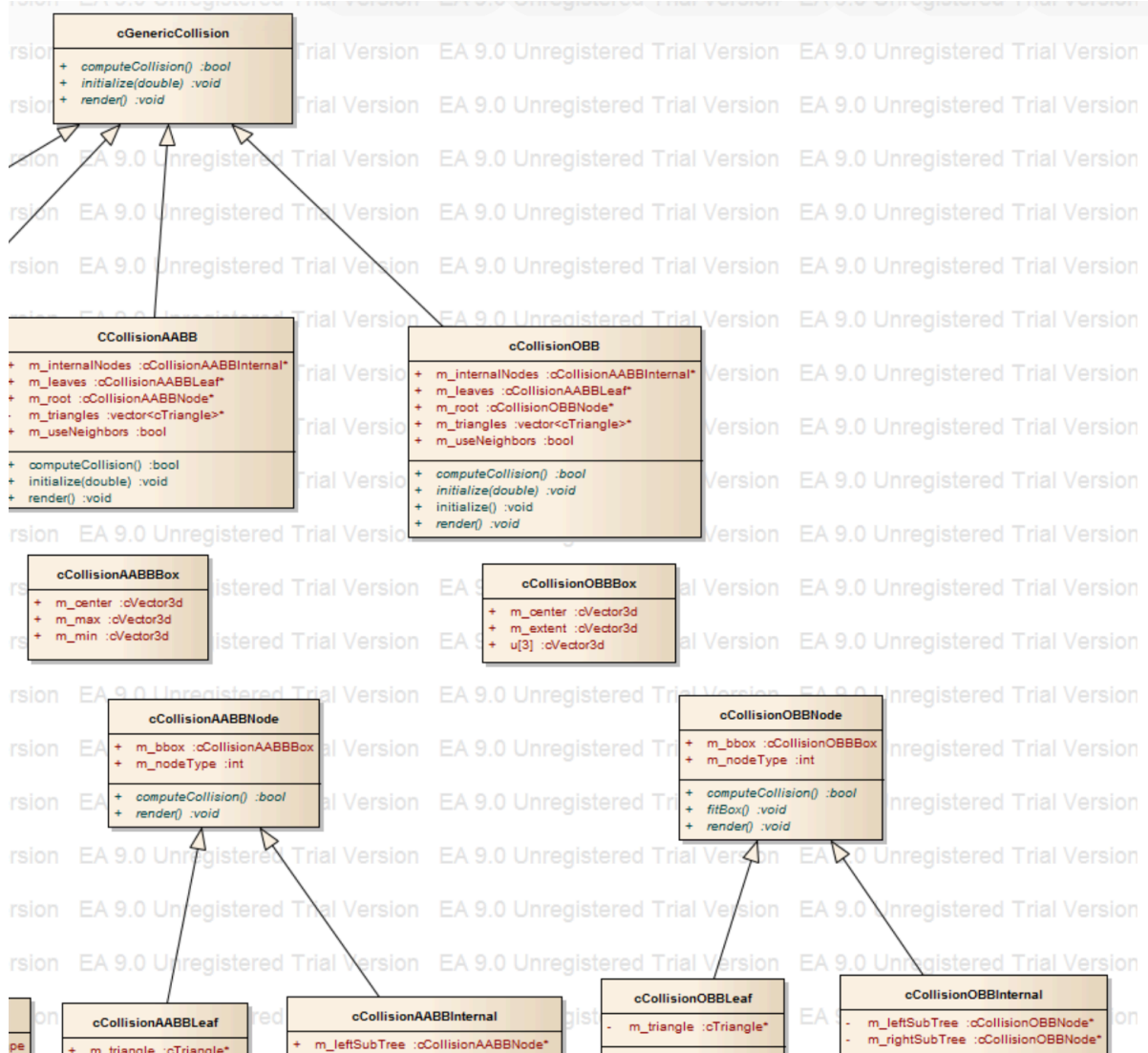
4. **Kiểm tra va chạm (Coarse Test & Fine Test):** Cài đặt hàm `computeCollision()` sử dụng thuật toán tìm kiếm theo chiều sâu (DFS) để duyệt cây OBB. Kiểm tra va chạm giữa một đoạn thẳng (đại diện cho vị trí HIP - Haptic Interface Point) và các node OBB (Coarse test). Nếu có va chạm với node lá, tiếp tục kiểm tra va chạm với tam giác bên trong (Fine test).
5. **Tích hợp vào cMesh:** Cập nhật lớp cMesh của Chai3D, thêm phương thức `createOBBCollisionDetector()` để người dùng có thể dễ dàng khởi tạo module OBB cho một đối tượng mesh.
6. **Hiển thị (Visualization):** Cài đặt hàm `render()` để vẽ các khung OBB trực quan trên màn hình OpenGL, hỗ trợ quá trình debug và kiểm tra kết quả.
7. **Tính năng nâng cao (Tùy chọn - Thêm điểm):** Cài đặt Local Search. Tận dụng thông tin va chạm từ khung hình trước (temporal coherence) để chỉ kiểm tra va chạm với các tam giác lân cận trong khung hình hiện tại, giúp giảm thiểu độ phức tạp tính toán.

3. Hướng dẫn thực hiện

3.1. Cấu trúc lớp đề xuất (Class Diagram)

Bạn nên thiết kế hệ thống lớp kế thừa cấu trúc hiện có của Chai3D. Ví dụ:

- `cCollisionOBB` kế thừa `cGenericCollision`.
- `cCollisionOBBNode`.
- `cCollisionOBBLeaf` và `cCollisionOBBInternal` kế thừa `cCollisionOBBNode`.
- `cCollisionOBBBox` (chứa center, 3 vector hướng, và 3 giá trị extent).



3.2. Gọi ý xây dựng OBB bằng PCA

Để tạo ra một OBB vừa vặn bao quanh tập các đỉnh tại một node:

1. **Lấy mẫu đỉnh:** Tập hợp tất cả các đỉnh của các tam giác thuộc node đó.
2. **Ma trận hiệp phương sai (Covariance Matrix):** Xây dựng ma trận covariance 3×3 từ tập các điểm đã lấy mẫu. Có thể viết các hàm hỗ trợ như CovarianceMatrix(...).
3. **Trị riêng và Vector riêng (Eigenvalues & Eigenvectors):** Tìm 3 vector riêng của ma trận covariance. 3 vector này (trực chuẩn hóa) sẽ là 3 trục tọa độ cục bộ (định hướng) của OBB. Các thuật toán như Jacobi rotation (SymSchur2, Jacobi) có thể được sử dụng.
4. **Tính toán Extent:** Chiếu tất cả các điểm lên 3 trục tọa độ vừa tìm được để tìm giá trị lớn nhất và nhỏ nhất trên mỗi trục. Tâm OBB và các nửa chiều dài (extent) sẽ được tính từ các giá trị cực đại/cực tiểu này. Hãy viết hàm ExtremePointAlongDirection(...) để hỗ trợ.

3.3. Tích hợp vào project

- Tạo các file .h và .cpp mới (ví dụ: CCollisionOBB.h, CCollisionOBB.cpp, CCollisionOBBCBox.h...) và đặt vào thư mục collisions của Chai3D.
- Khai báo các file header này trong chai3d.h.
- Chỉnh sửa lớp cMesh trong Chai3D để thêm hàm createOBBCollisionDetector(...).
- Sửa đổi file main.cpp của một ví dụ có sẵn (như ví dụ load file .obj, .3ds) để gọi hàm createOBBCollisionDetector thay vì AABB.

4. Thang điểm đánh giá (Tổng: 100 điểm + 10 điểm thưởng)

Bài tập sẽ được đánh giá dựa trên mức độ hoàn thiện của mã nguồn, tính chính xác của thuật toán và khả năng hiển thị trực quan.

Tiêu chí	Mô tả chi tiết	Điểm
1. Cấu trúc dữ liệu và Tích hợp cơ bản	Định nghĩa Class: Định nghĩa đúng cấu trúc cCollisionOBBCBox và các lớp Node (Internal, Leaf).	10
	Tích hợp Chai3D: Tích hợp thành công (compile không lỗi, gọi được hàm khởi tạo từ main.cpp thông qua cMesh).	10
2. Thuật toán Xây dựng Cây OBB	Logic chia tách: Cài đặt đúng logic chia tách (top-down) từ Root đến Leaf (vd: chia theo trục dài nhất).	10
	Tính toán OBB (PCA): Cài đặt thuật toán (Covariance Matrix, Eigenvectors) để tính toán chính xác hướng (orientation) và kích thước (extents) của OBB tại mỗi node.	20
3. Thuật toán Phát hiện va	Duyệt cây: Cài đặt đúng	10

chạm	thuật toán DFS duyệt cây computeCollision().	
	Kiểm tra giao cắt (Intersection test): Cài đặt chính xác phép kiểm tra giao cắt giữa Đoạn thẳng (Segment) và OBB (Ray-OBB/Segment-OBB). Chuyển tiếp đúng xuống tam giác ở node lá (Fine test).	15
4. Hiển thị (Visualization)	Render OBB: Cài đặt hàm render() sử dụng lệnh OpenGL vẽ khung dây (wireframe) của các OBB hiển thị đúng trên màn hình (đúng vị trí, góc xoay, kích thước). Hỗ trợ bật/tắt hiển thị.	15
5. Điểm thưởng / Nâng cao	Local Search & Đánh giá hiệu năng: Cài đặt thành công chiến lược Local Search để tối ưu. Thực hiện đo và so sánh tốc độ thực thi của giải thuật với các tập dữ liệu mesh có độ phức tạp khác nhau: 1.000, 10.000, 100.000 và 1.000.000 tam giác.	+10

Lưu ý: Nếu code bị lỗi haptic rendering (ví dụ: lực rung giật không ổn định - *instability*) như báo cáo mẫu đề cập nhưng phần collision detection logic vẫn đúng, sinh viên sẽ không bị trừ điểm nặng, tuy nhiên cần giải thích rõ lý do trong báo cáo bài tập. Các bạn nếu muốn thử giải thuật trên robot thì có thể đến phòng lab tại HCM hoặc Hà Nội để test xem độ ổn định của giải thuật và hiệu năng trên robot thật. Khuyến khích các bạn quay video khi test với robot - sẽ được cộng điểm